

JPA & HIBERNATE SEMINAR

Troubleshooting & Best Practices

Praxis-Leitfaden: Typische Fallstricke, Fehlermuster, Concurrency & Architekturregeln

Enterprise Java Persistence | Jakarta Persistence 3.0 & Hibernate ORM 6

Leitfaden-Struktur & Themenübersicht

1. Typische Fehlerbilder

- 1.1 LazyInitializationException & Proxies
- 1.2 N+1 Select Problem & JOIN FETCH
- 1.3 Entity Graphs (fetchgraph / loadgraph)
- 1.4 OptimisticLockException & @Version
- 1.5 Pessimistisches Locking & Concurrency
- 1.6 Transiente & Detached Entity Ausnahmen
- 1.7 Bean Validation & Constraints

2. Architecture & Best Practices

- 2.1 FetchType-Regeln (LAZY als Default)
- 2.2 Defensive Assoziations-Synchronisation
- 2.3 CascadeType & orphanRemoval
- 2.4 equals() & hashCode() Regeln bei Entities
- 2.5 Persistence Context & Batch Processing
- 2.6 Bulk Operations & Cache Invalidation
- 2.7 DTO-Projektionen & Constructor Expressions
- 2.8 Composite Keys (@EmbeddedId vs @IdClass)
- 2.9 AttributeConverter (@Converter)

3. Referenz- & Entscheidungstabellen

- 3.1 CascadeType-Matrix & Orphan Removal
- 3.2 Entity-Lifecycle Callbacks (@PrePersist etc.)
- 3.3 Checkliste für den Produktiveinsatz

1.1 LazyInitializationException

Symptom & Fehlermeldung

`org.hibernate.LazyInitializationException: could not initialize proxy [net.rentacar.model.Shop#1] - no Session`

Ursache & Mechanismen

- Zugriff auf eine LAZY-Relation bei einer Detached Entity (Session geschlossen / Tx beendet).
- Der Bytecode-Proxy kann ohne aktive Datenbankverbindung kein SQL nachladen.

Antipatterns (Vermeiden!):

- Globales Umstellen auf `FetchType.EAGER` (verschleppt Performance-Probleme).
- Open Session in View (OSIV) im Web/API-Layer (verschleiert unkontrollierte Queries).

Lösungsstrategien (Best Practice)

- 1. JPQL JOIN FETCH: Gezieltes Eager-Laden genau für diesen Abfragepfad.
- 2. JPA Entity Graphs: Deklarative oder dynamische Ladepläne definieren.
- 3. DTO Constructor Expressions: Flache Datentransfers direkt per JPQL projizieren.

```
SELECT s FROM Shop s JOIN FETCH s.vehicles WHERE s.id = :id
```

```
// DTO Projektion:
```

```
SELECT new net.rentacar.dto.ShopDto(s.name, size(s.vehicles)) FROM Shop s
```

1.2 Das N+1-Select-Problem & JOIN FETCH

Wie das Problem entsteht

- 1 initiale Abfrage lädt N Stammsätze (z.B. 50 Standorte):
- `SELECT * FROM SHOP`
- Bei Iteration über `shop.getVehicles()` feuert Hibernate für jeden einzelnen Shop ein separates `SELECT`:
- `SELECT * FROM VEHICLE WHERE SHOP_ID = 1`
`SELECT * FROM VEHICLE WHERE SHOP_ID = 2`
... (50x nachgeladen!)
- Ergebnis: $1 + N = 51$ SQL Queries statt 1 optimierter Query!

Lösung & Diagnose

1. JOIN FETCH Abfrage (1 einziger SQL-Join)

```
SELECT DISTINCT s FROM Shop s  
LEFT JOIN FETCH s.vehicles
```

2. Batch Fetching per @BatchSize

```
@OneToMany(mappedBy = "shop")  
@BatchSize(size = 25)  
private Set<Vehicle> vehicles;
```

- Diagnose: `hibernate.show_sql=true` und `hibernate.generate_statistics=true` aktivieren, um Statement-Zähler in Tests zu assertieren.

1.3 JPA Entity Graphs & Dynamisches Fetching

Warum Entity Graphs statt JOIN FETCH?

- Keine Query-Modifikation: JPQL bleibt einfach ('SELECT s FROM Shop s'), der Ladeplan wird dynamisch als Hint übergeben.
- Vermeidung des kartesischen Produkts: Mehrere JOIN FETCH auf Collections erzeugen Duplikate und MultipleBagFetchExceptions.
- Sauberes Entity-Mapping: Das Basis-Mapping bleibt durchgängig auf FetchType.LAZY.
- Zwei Modi als Query-Hint:
 - `jakarta.persistence.fetchgraph`:
Alle gelisteten Attribute = EAGER, alle anderen = LAZY (überschreibt statisches Mapping strikt).
 - `jakarta.persistence.loadgraph`:
Alle gelisteten Attribute = EAGER, alle anderen behalten ihr Mapping (statisches EAGER bleibt EAGER).

Programmatische & Deklarative Nutzung

```
// 1. Programmatischer EntityGraph
EntityGraph<Shop> graph = em.createEntityGraph(Shop.class);
graph.addAttributeNodes("carpool");
// Optional: Verschachtelter Subgraph
Subgraph<Vehicle> vehGraph = graph.addSubgraph("carpool");
vehGraph.addAttributeNodes("type");

List<Shop> shops = em.createQuery(
    "SELECT s FROM Shop s", Shop.class)
    .setHint("jakarta.persistence.fetchgraph", graph)
    .getResultList();

// 2. Deklarativ an der Entity (@NamedEntityGraph)
@NamedEntityGraph(
    name = "Shop.withCarpool",
    attributeNodes = @NamedAttributeNode("carpool")
)
@Entity
public class Shop { ... }
```

1.4 OptimisticLockException & @Version

Lost Updates & Concurrency-Konflikte

- Ohne Versionierung überschreibt die zuletzt committende Transaktion alle vorherigen Änderungen unbemerkt (Lost Update).
- Lösung mit @Version: Hibernate prüft bei jedem UPDATE die Versionsnummer:
- UPDATE VEHICLE SET ..., VERSION = 2 WHERE ID = 1 AND VERSION = 1
- Wurde die Zeile zwischenzeitlich verändert, liefert die DB 0 betroffene Zeilen -> Hibernate wirft `jakarta.persistence.OptimisticLockException`.

```
@Entity
public class Vehicle {
    @Id @GeneratedValue private Long id;
    @Version private Long version; // Optimistic Lock
}
```

Best Practices zur Ausnahmebehandlung

- 1. Transaktion sofort rollbacken: Nach einer `OptimisticLockException` ist der Persistence Context ungültig.
- 2. Retry-Strategie im Service-Layer: Transaktion neu starten, frische Entity aus der DB laden und Änderung erneut anwenden.
- 3. Konfliktanzeige für Endanwender: Den Benutzer informieren, dass der Datensatz parallel modifiziert wurde.

```
try {
    tx.commit();
} catch (RollbackException e) {
    if (e.getCause() instanceof OptimisticLockException) {
        // Rollback ausführen, neu laden & ggf. wiederholen
        tx.rollback();
        throw new ConcurrentModificationException("Datensatz wurde
geändert!");
    }
}
```

1.5 Concurrency: Pessimistisches Locking

Wann Optimistic Locking nicht reicht

- Hohe Schreibkollisionsrate: Wenn dutzende Benutzer gleichzeitig denselben Datensatz ändern (z.B. Ticketkontingent, Fahrzeugreservierung, Kontostand).
- Optimistic Locking führt hier zu ständigen Rollbacks und teuren Retries.
- Lösung: Pessimistic Locking (SELECT ... FOR UPDATE). Sperrt die Zeile physisch auf Datenbankebene gegen parallele Modifikationen.
- Modi:
 - • PESSIMISTIC_WRITE: Exklusiver Schreib-Lock (verhindert Read & Write anderer Locks).
 - • PESSIMISTIC_READ: Shared Read-Lock (verhindert gleichzeitige Modifikationen).

Code-Muster & Lock-Timeouts

```
// 1. Beim Laden direkt sperren (SELECT ... FOR UPDATE)
PessimisticVehicle v = em.find(
    PessimisticVehicle.class, 1L,
    LockModeType.PESSIMISTIC_WRITE);

// 2. Nachträglich in bestehender Entity sperren
em.lock(v, LockModeType.PESSIMISTIC_WRITE);

// 3. Lock-Timeout konfigurieren (z.B. 3 Sekunden)
TypedQuery<PessimisticVehicle> query = em.createQuery(
    "SELECT v FROM PessimisticVehicle v WHERE v.reserved = false",
    PessimisticVehicle.class)
    .setLockMode(LockModeType.PESSIMISTIC_WRITE)
    .setHint("jakarta.persistence.lock.timeout", 3000);
```

- Wichtig: Transaktionen mit pessimistischen Locks extrem kurz halten, um Deadlocks zu vermeiden!

1.6 Transiente & Detached Entity Ausnahmen

TransientPropertyValueException

IllegalStateException: Not-null property references a transient value
- transient instance must be saved before current operation

- Ursache: Eine gemanagte Entity verweist auf ein neu erzeugtes (transientes) Objekt, ohne dass CascadeType.PERSIST oder CascadeType.ALL aktiv ist.
- Lösung: Kindobjekt explizit mit em.persist() speichern oder Kaskadierung konfigurieren.

Detached Entity passed to persist

PersistenceException: detached entity passed to persist:
net.rentacar.model.Vehicle

- Ursache: Aufruf von em.persist(entity) auf einem Objekt, das bereits eine ID besitzt und aus einer früheren Session stammt.
- Lösung: Verwende em.merge(detachedEntity). Wichtig: merge() modifiziert nicht die übergebene Instanz, sondern liefert eine NEUE gemanagte Referenz zurück:
- Vehicle managed = em.merge(detachedVehicle);

1.7 Bean Validation & Constraints

ConstraintViolationException beim Flush

`jakarta.validation.ConstraintViolationException: Validation failed for [net.rentacar.model.User]`

- Zeitpunkt: JPA führt Bean Validation (@NotNull, @Size, @Pattern) standardmäßig automatisch beim PrePersist und PreUpdate (vor dem SQL Flush) aus.
- Gefahr: Fehler treten erst beim commit() oder manuellen flush() auf, weit entfernt von der setXxx()-Zuweisung!

Best Practices & Details auslesen

```
try {
    em.flush();
} catch (ConstraintViolationException e) {
    for (ConstraintViolation<?> v : e.getConstraintViolations()) {
        log.error("Feld '{}' {}: {}",
            v.getPropertyPath(), v.getMessage(),
            v.getInvalidValue());
    }
}
```

- Tipp: Validierungs-Annotationen steuern bei DDL-Generierung auch das DB-Schema (@NotNull -> NOT NULL Spalte).

1.8 @MappedSuperclass vs. Entity Vererbung

@MappedSuperclass (Technische Basisklasse)

- Keine eigene Tabelle in der Datenbank.
- Vererbt Attribute (z.B. @Id, @Version, createdAt, createdBy) an konkrete Unterklassen.
- Nicht polymorph abfragbar: 'SELECT b FROM BaseEntity b' ist UNZULÄSSIG!
- Ideal für Domain-Driven Design Basistypen und Audit-Felder.

```
@MappedSuperclass
public abstract class AbstractEntity {
    @Id @GeneratedValue private Long id;
    @Version private Long version;
}
```

@Inheritance (Fachliche Vererbung)

- Basisklasse ist eine echte @Entity mit Tabellenrepräsentation.
- Polymorphe Abfragen sind möglich: 'SELECT v FROM VehicleType v' liefert alle Pkw, Lkw und Motorräder.
- Strategien:
 - SINGLE_TABLE: 1 Tabelle mit @DiscriminatorColumn (schnell, Nullable Spalten).
 - JOINED: Normalisierte Tabellen je Subklasse (Constraint-sicher, Joins nötig).

2.1 FetchType & Assoziations-Strategie

Die JPA Default-Falle

- JPA Standard-Defaults:
 - • @OneToOne -> FetchType.EAGER (Gefährlich!)
 - • @ManyToOne -> FetchType.EAGER (Gefährlich!)
 - • @OneToMany -> FetchType.LAZY (Korrekt)
 - • @ManyToMany -> FetchType.LAZY (Korrekt)
- Problem: EAGER auf To-One führt zu automatischen SQL-Joins bei JEDEM find() oder SELECT, selbst wenn die Relation gar nicht benötigt wird.

Die goldene Regel: Alles auf LAZY

- Stelle ausnahmslos ALLE @ManyToOne und @OneToOne Relationen explizit auf FetchType.LAZY um!
- Lade benötigte Daten gezielt pro Use-Case über JOIN FETCH oder Entity Graphs nach.

```
@ManyToOne(fetch = FetchType.LAZY, optional = false)
@JoinColumn(name = "SHOP_ID")
private Shop shop; // Vorbildlich!
```

2.2 Defensive Assoziations-Synchronisation

Bidirektionale Beziehungen konsistent im Speicher halten

- JPA persistiert nur die 'Owning Side' (die Seite mit @JoinColumn). Die inverse Seite (mappedBy) wird für SQL ignoriert.
- Wird nur eine Seite im Java-Speicher gepflegt, ist der Objektgraph im selben Persistence Context inkonsistent!
- Best Practice: Schreibe defensive Hilfsmethoden in der Parent-Entity, die beide Seiten atomar synchronisieren.

```
public void addVehicle(Vehicle vehicle) {  
    this.carpool.add(vehicle);  
    vehicle.setShop(this); // Beide Seiten synchron halten!  
}
```

```
public void removeVehicle(Vehicle vehicle) {  
    this.carpool.remove(vehicle);  
    vehicle.setShop(null); // Fremdschlüssel aufheben  
}
```

2.3 CascadeType & orphanRemoval in der Praxis

orphanRemoval vs. CascadeType.REMOVE

- CascadeType.REMOVE: Löscht Kind-Datensätze nur dann, wenn das übergeordnete Elternobjekt mit em.remove(parent) gelöscht wird.
- orphanRemoval = true: Löscht ein Kindobjekt zusätzlich per SQL DELETE, wenn es aus der Java-Collection des Elternobjekts entfernt wird!
- Echter Parent-Child Lifecycle (Aggregat): Kindobjekte haben keine eigenständige Existenzberechtigung (z.B. RentalContract -> DamageRecords).
- Achtung: Niemals orphanRemoval auf @ManyToMany oder geteilten Stammdaten setzen!

Code-Beispiel: Sauberes Aggregat

```
@Entity
public class RentalContract {
    @OneToMany(mappedBy = "contract",
                cascade = CascadeType.ALL,
                orphanRemoval = true)
    private List<DamageRecord> damages = new ArrayList<>();

    public void removeDamage(DamageRecord damage) {
        damages.remove(damage);
        damage.setContract(null); // Führt zum SQL DELETE!
    }
}
```

- Testbar: contract.removeDamage(d) -> em.flush() -> SELECT COUNT(*) FROM DAMAGERECORD liefert 0!

2.4 Entity Identity: equals() & hashCode()

Das HashSet / Set-Problem

- Vor dem Persistieren ist `id == null` -> `hashCode() = 0`.
- Nach `em.persist()` wird eine ID generiert -> `hashCode()` ändert sich!
- Wurde die Entity zuvor in ein HashSet eingefügt, wird sie im Set nie wieder gefunden (Memory Leak / HashSet Invarianz gebrochen)!

Die Best-Practice Implementierung

```
@Override
public boolean equals(Object o) {
    if (this == o) return true;
    if (o == null || getClass() != o.getClass()) return false;
    Vehicle that = (Vehicle) o;
    // ID-Vergleich nur wenn beide IDs nicht null sind
    return id != null && Objects.equals(id, that.id);
}

@Override
public int hashCode() {
    // Fester Hashcode für alle Instanzen dieser Klasse!
    return getClass().hashCode();
}
```

2.5 Persistence Context & Batch Processing

Speicherbereinigung bei Massendaten-Verarbeitung

- Jede geladene oder persistierte Entity verbleibt dauerhaft im 1st-Level-Cache (EntityManager).
- Beim Import von 100.000 Datensätzen führt dies zwangsläufig zu einem OutOfMemoryError.
- Lösung: Regelmäßiges Spülen (flush()) und Entleeren (clear()) des Caches in Batches (z.B. alle 50-100 Einträge).

```
for (int i = 0; i < vehicles.size(); i++) {  
    em.persist(vehicles.get(i));  
    if (i > 0 && i % 50 == 0) {  
        em.flush(); // Schreibt anstehende INSERTs in die DB  
        em.clear(); // Leert den 1st-Level-Cache vollständig!  
    }  
}  
tx.commit();
```

2.6 Bulk Operations (executeUpdate) & 1st-Level Cache

Die Stale-State-Falle bei Massenupdates

- JPQL UPDATE und DELETE (query.executeUpdate()) werden direkt als SQL an die Datenbank gesendet.
- Wichtig: Der Persistence Context (1st-Level-Cache) weiß nichts von dieser Massenänderung!
- Bereits zuvor geladene Managed Entities behalten ihren alten Speicherstand (Stale Data).
- Gefahr: Folgezugriffe arbeiten auf veralteten Objekten oder überschreiben den DB-Stand beim nächsten Flush!

Das 3-Schritt-Muster (Flush -> Execute -> Clear)

```
// 1. Ungespeicherte Änderungen in die DB flushen
em.flush();

// 2. Massen-Update direkt auf DB ausführen
int updated = em.createQuery(
    "UPDATE BulkVehicle v SET v.dailyRate = v.dailyRate * 1.1 " +
    "WHERE v.active = true")
    .executeUpdate();

// 3. Cache leeren, damit Entities frisch geladen werden!
em.clear(); // oder gezielt em.refresh(entity);
```

- Regel: Nach executeUpdate() immer em.clear() aufrufen, bevor weiter mit Entities gearbeitet wird.

2.7 DTO-Projektionen (Constructor Expressions)

Reine Lese-Views ohne Entity-Overhead projizieren

- Für REST-APIs, Tabellenansichten und Reports müssen selten vollständige Managed Entities geladen werden.
- Constructor Expressions (SELECT new ...) instanziiieren schlanke, unveränderliche Java-Records oder DTOs direkt im SELECT.
- Kein Dirty Checking Overhead, kein First-Level-Cache, keine LazyInitializationException im Web-Layer!

// DTO Definition (z.B. Java Record):

```
public record VehicleSummaryDto(String brand, String model, double dailyRate) {}
```

// JPQL Query mit vollqualifiziertem Klassennamen:

```
List<VehicleSummaryDto> dtos = em.createQuery(  
    "SELECT new net.rentacar.dto.VehicleSummaryDto(v.brand, v.modell, v.dailyRate) " +  
    "FROM Vehicle v WHERE v.active = true", VehicleSummaryDto.class)  
    .getResultList();
```

2.8 Composite Keys: @EmbeddedId vs. @IdClass

@EmbeddedId (Empfohlener Standard)

- Kompaktes Schlüsselobjekt mit @Embeddable, Serializable und equals/hashCode.
- Klare Objektorientierung: Entity hat genau 1 ID-Attribut:
- `@EmbeddedId private BookingId id;`
- Einfaches und typsicheres Finden:
- `em.find(Booking.class, new BookingId('R-01', 'BER'));`
- Wartungsfreundlich und robust bei Refactorings.

@IdClass (Historische Alternative)

- Die Schlüsselattribute werden flach in der Entity wiederholt:
- `@Id private String contractNumber;`
- `@Id private String branchCode;`
- Zusätzlich separate IdClass mit identischen Attributnamen nötig.
- Nachteile: Redundanz, flaches Modell, unübersichtlich bei Beziehungen.
- Fazit: In modernen Projekten immer @EmbeddedId bevorzugen!

2.9 Custom Type Mapping mit AttributeConverter

Die Risiken klassischer Enum-Mappings

- @Enumerated(EnumType.ORDINAL): Speichert 0, 1, 2... Ein Umsortieren der Enum-Konstanten führt zu unbemerkter, massiver Datenkorruption!
- @Enumerated(EnumType.STRING): Speichert 'ELECTRIC', 'PETROL'... Braucht viel DB-Speicherplatz und bricht bei Java-Umbenennungen.
- Lösung: JPA AttributeConverter (JPA 2.1+).

Implementierung mit @Converter

```
@Converter(autoApply = true)
public class FuelTypeConverter
    implements AttributeConverter<FuelType, String> {

    public String convertToDatabaseColumn(FuelType type) {
        return type != null ? type.getCode() : null; // z.B. "E", "D"
    }

    public FuelType convertToEntityAttribute(String code) {
        return FuelType.fromCode(code);
    }
}
```

- Vorteil: DB speichert kompakte Einzelzeichen, Java nutzt typsicheres Enum. autoApply = true bindet den Konverter global!

3.1 CascadeType Referenztabelle

CascadeType	JPA Operation	Wirkung auf Kind-Entities	Empfohlener Einsatz
PERSIST	em.persist()	Transiente Kinder werden automatisch persistiert	1:1, 1:N Aggregate (z.B. Rechnung -> Posten)
MERGE	em.merge()	Detached Kinder werden mit aktualisiert	Web-Formulare mit verschachtelten Daten
REMOVE	em.remove()	Löscht alle Kinder beim Löschen des Parents	Strikte 1:1 und echte Parent-Child Aggregate
REFRESH	em.refresh()	Lädt Kind-Zustand aus DB neu	Selten; bei extern geänderten DB-Triggern
DETACH	em.detach()	Entfernt auch Kinder aus dem Cache	Manuelle Session-Bereinigung

Achtung: CascadeType.ALL oder REMOVE auf @ManyToMany führt dazu, dass beim Löschen eines Datensatzes verknüpfte Stammdaten (z.B. Fahrer, Kategorien) gelöscht werden!

3.2 Entity Lifecycle Callbacks & Listener

Annotation	Ausführungszeitpunkt	Praxiseinsatz / Anwendungsfall
@PrePersist	Vor dem ersten INSERT	Setzen von createdAt = Instant.now(), Default-Werten
@PostPersist	Nach dem erfolgreichen INSERT	Audit-Logging mit generierter ID, Event-Publishing
@PreUpdate	Vor dem SQL UPDATE	Aktualisieren von updatedAt = Instant.now()
@PostUpdate	Nach dem SQL UPDATE	Cache-Invalidierung, Benachrichtigungen
@PreRemove	Vor dem SQL DELETE	Konsistenzprüfung, Validierung von Abhängigkeiten
@PostRemove	Nach dem SQL DELETE	Aufräumarbeiten im Dateisystem, Audit-Log
@PostLoad	Nach dem SELECT aus der DB	Initialisieren transienter (@Transient) Berechnungsfelder

JPA Best-Practice Checkliste für den Produktiveinsatz

1. Mapping & Beziehungen

- Alle @ManyToOne / @OneToOne explizit auf LAZY.
- Defensive Helper-Methoden (add/remove) in Owning Entities.
- orphanRemoval=true nur für echte Eltern-Kind-Aggregate.
- Kein CascadeType.ALL oder REMOVE auf ManyToMany.
- AttributeConverter für saubere Enums & Custom Types.

2. Identity & Concurrency

- equals/hashCode mit Business Key oder konstantem Hash.
- @Version für optimistische Sperren in Concurrency-Szenarien.
- Pessimistic Locking (SELECT FOR UPDATE) für Hotspots.
- @EmbeddedId als Standard für zusammengesetzte Schlüssel.

3. Abfragen & Performance

- JOIN FETCH oder Entity Graphs gegen N+1 Selects.
- DTO Constructor Expressions für Lese-Views & APIs.
- Batch Processing: flush() & clear() alle 50-100 Datensätze.
- Nach executeUpdate() immer em.clear() aufrufen!